

```

"""
fase0_diagnostico.py
=====

FASE 0 - Diagnostico inicial del dataset.

No elimina nada. Su unico objetivo es dejar por escrito el estado del dataset
ANTES de tocarlo, de modo que cualquier decision posterior pueda contrastarse
contra una fotografia del punto de partida.

Produce tres tablas:

* ``diagnostico`` : una fila por columna con su perfil completo.
* ``general`` : metricas de cabecera del dataset (filas, columnas, panel).
* ``candidatas_exclusion_temprana`` : columnas que ya se ven problematicas.

La "exclusion temprana" es un AVISO, no una eliminacion: la eliminacion formal
ocurre en la fase 1 con sus umbrales explicitos. Separarlo evita que el
diagnostico se convierta en una decision oculta.
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .config import ConfigPipeline
from .logging_utils import obtener_logger
from .validaciones import ReporteValidacion, calcular_varianza_panel

LOGGER = obtener_logger("fase0")

#: Percentiles exigidos por la especificacion del proyecto.
PERCENTILES = (0.25, 0.50, 0.75, 0.90, 0.99)

def perfilar_columna(
    df: pd.DataFrame, col: str, tipo: str, cfg: ConfigPipeline
) -> dict:
    """Construye el perfil estadistico completo de una columna."""
    serie = df[col]
    n = len(serie)

    n_nulos = int(serie.isna().sum())
    pct_nulos = n_nulos / n if n else np.nan

    # --- Ceros: solo tiene sentido en variables numericas -----
    if tipo in ("NUMERICA", "BOOLEANA"):
        numerica = pd.to_numeric(serie, errors="coerce")
        n_ceros = int((numerica == 0).sum())
    else:
        # En texto se considera "cero" el vacio o el literal "0": es el
        # equivalente semantico de ausencia de contenido.
        texto = serie.astype(str).str.strip()
        n_ceros = int(((texto == "") | (texto == "0")).sum())
        numerica = pd.Series(dtype=float)

    pct_ceros = n_ceros / n if n else np.nan

    perfil: dict = {
        "columna": col,
        "rol": cfg.rol_de(col),
        "tipo_inferido": tipo,
        "dtype_pandas": str(serie.dtype),
        "n_filas": n,
        "n_nulos": n_nulos,
        "pct_nulos": pct_nulos,
        "n_ceros": n_ceros,
        "pct_ceros": pct_ceros,
        "pct_ceros_mas_nulos": (n_ceros + n_nulos) / n if n else np.nan,
        "n_unicos": int(serie.nunique(dropna=True)),
    }

```

```

    "pct_unicos": float(serie.nunique(dropna=True) / n) if n else np.nan,
    "memoria_mb": float(serie.memory_usage(deep=True) / 1024**2),
}

# --- Valor modal y su dominancia -----
no_nulos = serie.dropna()
if len(no_nulos):
    conteo = no_nulos.value_counts()
    perfil["valor_mas_frecuente"] = str(conteo.index[0])[:120]
    perfil["frecuencia_valor_mas_frecuente"] = int(conteo.iloc[0])
    perfil["pct_valor_mas_frecuente"] = float(conteo.iloc[0] / len(no_nulos))
else:
    perfil["valor_mas_frecuente"] = ""
    perfil["frecuencia_valor_mas_frecuente"] = 0
    perfil["pct_valor_mas_frecuente"] = np.nan

# --- Estadisticos numericos -----
campos_num = ["media", "desviacion_estandar", "varianza", "minimo", "maximo",
              "p25", "p50", "p75", "p90", "p99", "iqr", "rango",
              "coef_variacion", "asimetria", "curtosis",
              "var_within", "var_between", "icc_panel"]
for c in campos_num:
    perfil[c] = np.nan

if tipo in ("NUMERICA", "BOOLEANA") and numerica.notna().sum() > 0:
    v = numerica.dropna().astype(float)
    perfil["media"] = float(v.mean())
    perfil["desviacion_estandar"] = float(v.std(ddof=1)) if len(v) > 1 else 0.0
    perfil["varianza"] = float(v.var(ddof=1)) if len(v) > 1 else 0.0
    perfil["minimo"] = float(v.min())
    perfil["maximo"] = float(v.max())

    q = v.quantile(list(PERCENTILES))
    perfil["p25"] = float(q.loc[0.25])
    perfil["p50"] = float(q.loc[0.50])
    perfil["p75"] = float(q.loc[0.75])
    perfil["p90"] = float(q.loc[0.90])
    perfil["p99"] = float(q.loc[0.99])
    perfil["iqr"] = perfil["p75"] - perfil["p25"]
    perfil["rango"] = perfil["maximo"] - perfil["minimo"]

    media = perfil["media"]
    perfil["coef_variacion"] = (
        float(perfil["desviacion_estandar"] / abs(media)) if abs(media) > 1e-12 else np.nan
    )
    if len(v) > 3:
        perfil["asimetria"] = float(v.skew())
        perfil["curtosis"] = float(v.kurtosis())

# Descomposicion de varianza propia del panel.
if cfg.columna_id in df.columns:
    w, b, icc = calcular_varianza_panel(numerica, df[cfg.columna_id])
    perfil["var_within"] = w
    perfil["var_between"] = b
    perfil["icc_panel"] = icc

return perfil

def _motivo_exclusion_temprana(fila: pd.Series, cfg: ConfigPipeline) -> str:
    """Determina si una columna ya se ve problematica y por que (solo aviso)."""
    motivos: list[str] = []
    u = cfg.umbbral_ceros_nulos_efectivo

    if fila["rol"] != "CANDIDATA":
        return ""
    if fila["n_unicos"] <= 1:
        motivos.append("constante (1 unico valor)")
    if pd.notna(fila["pct_ceros_mas_nulos"]) and fila["pct_ceros_mas_nulos"] >= u:
        motivos.append(f"ceros+nulos {fila['pct_ceros_mas_nulos']:.1%} >= {u:.0%}")
    if pd.notna(fila["pct_nulos"]) and fila["pct_nulos"] >= u:

```

```

        motivos.append(f"nulos {fila['pct_nulos']:.1%} >= {u:.0%}")
    if pd.notna(fila["desviacion_estandar"]) and fila["desviacion_estandar"] <= cfg.umbral_std_minimo:
        motivos.append("desviacion estandar ~ 0")
    if pd.notna(fila["pct_valor_mas_frecuente"]) and fila["pct_valor_mas_frecuente"] >= cfg.umbral_dominancia:
        motivos.append(f"categoria dominante {fila['pct_valor_mas_frecuente']:.1%}")
    if fila["tipo_inferido"] == "CATEGORICA" and fila["pct_unicos"] > 0.90:
        motivos.append("cardinalidad casi unica (parece identificador)")
    return "; ".join(motivos)

def ejecutar(
    df: pd.DataFrame,
    cfg: ConfigPipeline,
    tipos: dict[str, str],
    rep_validacion: ReporteValidacion,
) -> dict[str, pd.DataFrame]:
    """Ejecuta la fase 0 y devuelve las tablas del diagnostico."""
    LOGGER.info("=" * 78)
    LOGGER.info("FASE 0 - DIAGNOSTICO INICIAL DEL DATASET")
    LOGGER.info("=" * 78)

    filas = [perfilar_columna(df, c, tipos.get(c, "CATEGORICA"), cfg) for c in df.columns]
    diagnostico = pd.DataFrame(filas)

    diagnostico["motivo_exclusion_temprana"] = diagnostico.apply(
        lambda f: _motivo_exclusion_temprana(f, cfg), axis=1
    )
    diagnostico["flg_candidata_exclusion_temprana"] = (
        diagnostico["motivo_exclusion_temprana"].str.len() > 0
    ).astype(int)

    # --- Varianza cero / casi cero -----
    diagnostico["flg_varianza_cero"] = (
        (diagnostico["varianza"].fillna(-1) <= cfg.umbral_std_minimo)
        & (diagnostico["rol"] == "CANDIDATA")
    ).astype(int)

    # -----
    # Tabla general de cabecera
    # -----
    n_filas, n_cols = df.shape
    candidatas = diagnostico[diagnostico["rol"] == "CANDIDATA"]

    general = pd.DataFrame(
        [
            ("Filas totales", n_filas, "Observaciones del panel (entidad x periodo)."),
            ("Columnas totales", n_cols, "Incluye target, id, tiempo y candidatas."),
            ("Columna target", cfg.columna_target, "Variable a predecir (parametro columna_target)."),
            ("Columna id", cfg.columna_id, "Identificador de entidad (parametro columna_id)."),
            ("Columna tiempo", cfg.columna_tiempo, "Dimension temporal (parametro columna_tiempo)."),
            ("Tipo de target detectado", rep_validacion.tipo_target, "Determina como se calculan IV y Gini."),
            ("Entidades distintas", rep_validacion.n_entidades, "Cardinalidad de la dimension transversal."),
            ("Periodos distintos", rep_validacion.n_periodos, "Cardinalidad de la dimension temporal."),
            ("Panel balanceado", "SI" if rep_validacion.panel_balanceado else "NO",
             "Balanceado = toda entidad observada en todo periodo."),
            ("Duplicados en llave id+tiempo", rep_validacion.duplicados_llave,
             "Debe ser 0: el panel exige una observacion por entidad y periodo."),
            ("Columnas candidatas a evaluar", len(candidatas),
             "Total menos target, id, tiempo y exclusiones manuales."),
            ("Columnas numericas", int((diagnostico["tipo_inferido"] == "NUMERICA").sum()), "Tipo inferido."),
            ("Columnas categoricas", int((diagnostico["tipo_inferido"] == "CATEGORICA").sum()), "Tipo inferido."),
            ("Columnas booleanas", int((diagnostico["tipo_inferido"] == "BOOLEANA").sum()), "Tipo inferido."),
            ("Columnas fecha", int((diagnostico["tipo_inferido"] == "FECHA").sum()), "Tipo inferido."),
            ("Celdas nulas totales", int(df.isna().sum().sum()),
             f"{df.isna().sum().sum() / max(n_filas * n_cols, 1):.4%} del total de celdas."),
            ("Columnas con algun nulo", int((diagnostico["n_nulos"] > 0).sum()), "Cobertura del problema de nulos."),
            ("Columnas con varianza cero", int(diagnostico["flg_varianza_cero"].sum()),
             "Candidatas sin dispersion alguna."),
            ("Columnas marcadas para exclusion temprana",
             int(diagnostico["flg_candidata_exclusion_temprana"].sum()),
             "Aviso del diagnostico; la eliminacion formal ocurre en la fase 1."),
        ]
    )

```

```

        ("Memoria del dataset (MB)", round(float(df.memory_usage(deep=True).sum() / 1024**2), 2), "Uso en RAM."
    ],
    columns=["metrica", "valor", "comentario"],
)

# -----
# Distribucion del target en el tiempo (util para leer el panel)
# -----
try:
    y_num = pd.to_numeric(df[cfg.columna_target], errors="coerce")
    por_periodo = (
        df.assign(_y=y_num)
        .groupby(cfg.columna_tiempo, observed=True)
        .agg(n_observaciones=("_y", "size"),
            n_entidades=(cfg.columna_id, "nunique"),
            target_medio=("_y", "mean"),
            target_nulos=("_y", lambda s: int(s.isna().sum()))
        ).reset_index()
    )
    por_periodo[cfg.columna_tiempo] = por_periodo[cfg.columna_tiempo].astype(str)
except Exception as exc: # noqa: BLE001 - es informativo, no debe romper
    LOGGER.warning("No se pudo construir la distribucion del target por periodo: %s", exc)
    por_periodo = pd.DataFrame()

exclusion_temprana = (
    diagnostico.loc[
        diagnostico["flg_candidata_exclusion_temprana"] == 1,
        ["columna", "tipo_inferido", "pct_nulos", "pct_ceros", "pct_ceros_mas_nulos",
         "n_unicos", "desviacion_estandar", "pct_valor_mas_frecuente",
         "motivo_exclusion_temprana"],
    ]
    .sort_values("pct_ceros_mas_nulos", ascending=False)
    .reset_index(drop=True)
)

LOGGER.info("Dataset: %d filas x %d columnas | %d candidatas a evaluar.",
            n_filas, n_cols, len(candidatas))
LOGGER.info("Panel: %d entidades x %d periodos (%s).", rep_validacion.n_entidades,
            rep_validacion.n_periodos,
            "balanceado" if rep_validacion.panel_balanceado else "desbalanceado")
LOGGER.info("Columnas marcadas para exclusion temprana (aviso): %d.", len(exclusion_temprana))

return {
    "diagnostico": diagnostico,
    "general": general,
    "target_por_periodo": por_periodo,
    "exclusion_temprana": exclusion_temprana,
}

```